IBM Databases and SQL for Data Science with Python Course | Coursera

Summary

Module one

Basic SQL

Introduction to Databases

SQL (Structured Query Language) is a language used for querying and managing data in relational databases. A **database** is a system for storing and organizing data efficiently, allowing quick access and modifications. Data, which can be in the form of words, numbers, or images, is one of the most valuable assets for businesses and is stored securely in databases.

A **relational database** organizes data in tables (similar to spreadsheets) with rows and columns, where columns represent attributes (e.g., name, email, city) and rows represent records. **Relational Database Management Systems (RDBMS)** provide tools for accessing, storing, and organizing data. Examples of RDBMS include **MySQL, Oracle Database, DB2 Warehouse, and DB2 on Cloud**.

There are **five fundamental SQL commands** used in databases:

1. **CREATE** – To create a new table.
2. **INSERT** – To add data into a table.
3. **SELECT** – To retrieve data from a table.
4. **UPDATE** – To modify existing data.
5. **DELETE** – To remove data from a table.

SQL plays a crucial role in industries such as banking, healthcare, and transportation by ensuring structured and efficient data management.

SELECT Statement

The **SELECT statement** is used to retrieve data from a relational database table. It is part of the **Data Manipulation Language (DML)** and allows users to view stored data. The result of executing a **SELECT** query is called a **result set** or **result table**.

Basic Syntax of SELECT Statement:

- Retrieve all columns from a table: `SELECT * FROM table_name;`
- Retrieve specific columns from a table:

```
SELECT column1, column2 FROM table_name;
```

Using the WHERE Clause

The **WHERE clause** is used to filter records based on specific conditions. It requires a **predicate**, which is a condition that evaluates to **TRUE, FALSE, or UNKNOWN**.

- Example: Retrieve the title of the book with `book_id = 'B1'`

```
SELECT book_id, title FROM book WHERE book_id = 'B1';
```

Comparison Operators in SQL

The WHERE clause supports various **comparison operators**:

- = (Equal to)
- > (Greater than)
- < (Less than)
- >= (Greater than or equal to)
- <= (Less than or equal to)
- != or <> (Not equal to)

By using the **SELECT statement**, **WHERE clause**, and **comparison operators**, you can efficiently retrieve and filter data from a relational database.

COUNT, DISTINCT, LIMIT

In SQL, there are several **useful expressions** that enhance the functionality of the **SELECT statement**. These include **COUNT, DISTINCT, and LIMIT**.

1. COUNT – Counting Rows in a Table

The **COUNT** function returns the number of rows that match a query's criteria.

- **Count all rows in a table:**

```
SELECT COUNT(*) FROM table_name;
```

- **Count rows where a condition is met (e.g., number of medal winners from Canada):**

```
SELECT COUNT(COUNTRY) FROM MEDALS WHERE COUNTRY = 'CANADA';
```

2. DISTINCT – Removing Duplicates

The **DISTINCT** keyword is used to retrieve unique values from a column, eliminating duplicates.

- **Retrieve unique countries that won a gold medal:**

```
SELECT DISTINCT COUNTRY FROM MEDALS WHERE MEDALTYPE = 'GOLD';
```

3. LIMIT – Restricting the Number of Rows

The **LIMIT** clause restricts the number of rows retrieved from the database, which is useful when dealing with large datasets.

- **Retrieve the first 10 rows from a table:**

```
SELECT * FROM table_name LIMIT 10;
```

- **Retrieve 5 rows of medals from the year 2018:**

```
SELECT * FROM MEDALS WHERE YEAR = 2018 LIMIT 5;
```

INSERT Statement

The **INSERT** statement is used to add new rows to a **relational database table**. It is part of the **Data Manipulation Language (DML)**, which allows users to read and modify data.

Syntax of INSERT Statement

```
INSERT INTO table_name (column1, column2, column3, ...)
VALUES (value1, value2, value3, ...);
```

- `table_name` – The name of the table.
- `(column1, column2, column3, ...)` – The list of columns to insert data into.
- `VALUES` – The clause that specifies the values to be inserted.

Methods to Insert Data

1. **Inserting a Single Row**

```
INSERT INTO author (author_id, last_name, first_name, email,
city, country)
VALUES ('A1', 'Chong', 'Raul', 'RFC@IBM.com', 'Toronto', 'CA');
```

   - This inserts a single row into the **author** table.
   - The number of values **must match** the number of specified columns.

2. **Inserting Multiple Rows at Once**

```
INSERT INTO author (author_id, last_name, first_name, email,
city, country)
VALUES
('A1', 'Chong', 'Raul', 'RFC@IBM.com', 'Toronto', 'CA'),
('A2', 'Ahuja', 'Rav', 'RA@IBM.com', 'New York', 'US');
```

- Multiple rows are inserted in a **single query** by separating values with commas.

UPDATE and DELETE Statements

The **UPDATE** and **DELETE** statements are used to **modify and remove data** in a relational database table. Both statements are part of the **Data Manipulation Language (DML)** and rely on the **WHERE clause** to specify which rows should be affected.

---

1. The UPDATE Statement – Modifying Data

The **UPDATE** statement alters existing data in a table.

Syntax:

```
UPDATE table_name
SET column1 = value1, column2 = value2
WHERE condition;
```

- **`table_name`** – The name of the table.
- **`SET`** – Defines the columns and new values to be updated.
- **`WHERE condition`** – Specifies which rows should be updated (if omitted, **all rows** will be updated).

Example: Updating a Specific Row

```
UPDATE AUTHOR
SET LASTNAME = 'KATTA', FIRSTNAME = 'LAKSHMI'
WHERE AUTHOR_ID = 'A2';
```

- This updates the **FIRSTNAME** and **LASTNAME** of the **AUTHOR_ID = 'A2'** row.

🔴 **Warning:** Omitting the **WHERE** clause will update **all rows** in the table.

---

2. The DELETE Statement – Removing Data

The **DELETE** statement removes **one or more rows** from a table.

Syntax:

```
DELETE FROM table_name
WHERE condition;
```

- `table_name` – The name of the table.
- `WHERE condition` – Specifies which rows should be deleted (if omitted, **all rows** will be deleted).

Example: Deleting Specific Rows

```
DELETE FROM AUTHOR
WHERE AUTHOR_ID IN ('A2', 'A3');
```

- This removes rows where **AUTHOR_ID** is **A2** or **A3**.

🔴 **Warning:** Omitting the **WHERE** clause will **delete all rows** in the table.

Key Takeaways:

- The **UPDATE statement** modifies data in a table.
- The **DELETE statement** removes data from a table.
- The **WHERE clause** is crucial to **target specific rows**; otherwise, **all rows** will be affected.

Summary: Basic SQL

- The Data Manipulation Language (DML) statements read and modify data.
- The search condition of the WHERE clause uses a predicate to refine the search.
- The SQL retrieves specific data from databases.
- The COUNT, DISTINCT, and LIMIT are expressions used with SELECT statements.
- The real-world applications of SELECT statements.
- The INSERT, UPDATE, and DELETE are DML statements for populating and changing tables.

Introduction to Relational Databases and Tables

Relational Database Concepts

1. The Relational Model & Its Advantages

The **relational model** is widely used because it provides:
✅ **Logical data independence** (data structure changes without affecting applications)
✅ **Physical data independence** (storage changes without affecting the database

design)
✅ **Physical storage independence** (data stored efficiently without affecting queries)

---

2. The Entity-Relationship (ER) Model

The **Entity-Relationship (ER) model** is a **tool for designing relational databases**.

- **Entity**: An independent object, such as **Book**, **Author**, or **Borrower**. Represented as a **rectangle** in ER diagrams.
- **Attribute**: Characteristics of an entity, such as **Book Title**, **Year Published**, etc. Represented as an **oval** in ER diagrams.

Example: Library Database

📚 **Entities**: Book, Author, Borrower, Loan, Copy
📌 **Attributes for Book**: Title, Edition, Year, ISBN

---

3. Mapping Entities to Tables

Step 1: Convert Entities to Tables

Each **entity** in the ER diagram becomes a **table** in the relational database.
Example:

- **Book Entity → BOOK Table**
- **Author Entity → AUTHOR Table**

Step 2: Convert Attributes to Columns

Each **attribute** becomes a **column** in the table.
Example:

- **Book Table Columns**: `book_id`, `title`, `edition`, `year`, `ISBN`

Step 3: Assign Data Types

Each column stores different **data types**, such as:

- **Character Data**: `CHAR`, `VARCHAR` (for text like book titles)
- **Numeric Data**: `INTEGER`, `DECIMAL` (for numbers like edition & year)
- **Date/Time Data**: `DATE`, `TIMESTAMP` (for loan records)

---

4. Primary Keys & Foreign Keys

Primary Key (PK)

A **primary key** uniquely identifies each **row** in a table.

- Example: `book_id` is the **PK** for the **Book** table.
- **Prevents duplicate records** and ensures **data integrity**.

Foreign Key (FK)

A **foreign key** is a **primary key from another table**, used to create **relationships**.

- Example: The **Author table** has `author_id` as **PK**, and the **Book table** may have `author_id` as an **FK**.

📌 **Primary keys help enforce uniqueness, while foreign keys establish table relationships.**

Key Takeaways

✓ **Entities** become **tables**, and **attributes** become **columns**.
✓ **Common data types** include **VARCHAR, CHAR, INTEGER, DECIMAL, DATE, TIMESTAMP**.
✓ **Primary Keys (PK)** uniquely identify rows & prevent duplicates.
✓ **Foreign Keys (FK)** define relationships between tables.
✓ **Relational databases** provide **data independence** and efficient storage.

Types of SQL statements (DDL vs. DML)

1️⃣ **Data Definition Language (DDL)** – Defines and manages database structures.
2️⃣ **Data Manipulation Language (DML)** – Works with the data inside tables.

---

1. Data Definition Language (DDL)

DDL statements are used to **define, modify, or delete database objects** like tables.
Common DDL statements include:

DDL Statement	Function
CREATE	Creates tables and defines columns.
ALTER	Modifies table structure (adds/drops columns, changes data types).
TRUNCATE	Deletes all rows from a table **without** removing the table itself.
DROP	Deletes a table **completely**, including its structure.

- **Example:**

```
CREATE TABLE Authors (
    author_id INT PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    email VARCHAR(100)
);
```

---

2. Data Manipulation Language (DML)

DML statements **insert, retrieve, update, and delete** data from tables.
They are also known as **CRUD operations** (Create, Read, Update, Delete).

DML Statement	Function
INSERT	Adds new rows to a table.
SELECT	Reads (retrieves) data from a table.
UPDATE	Modifies existing rows in a table.
DELETE	Removes specific rows from a table.

- **Example:**

```
INSERT INTO Authors (author_id, first_name, last_name, email)
VALUES (1, 'Raul', 'Chong', 'raul@example.com');
```

Key Differences Between DDL & DML

Feature	DDL (Definition)	DML (Manipulation)
Purpose	Defines/modifies database objects	Works with the data inside tables
Operations	CREATE, ALTER, DROP, TRUNCATE	INSERT, SELECT, UPDATE, DELETE
Effect	Affects table structure	Affects data inside tables
Rollback?	❌ **Cannot be rolled back** (permanent changes)	✅ **Can be rolled back** (with transactions)

Key Takeaways

✓ **DDL** modifies **database objects** like tables (**CREATE, ALTER, DROP**).
✓ **DML** manipulates **data** inside tables (**INSERT, SELECT, UPDATE, DELETE**).
✓ DDL changes are **permanent**, while DML changes can be **rolled back** using transactions.

CREATE TABLE Statement

The **CREATE TABLE** statement is a **Data Definition Language (DDL)** command used to create **tables** in a relational database. It defines the **entity name (table name) and attributes (columns with data types and constraints).**

---

1. Syntax of CREATE TABLE

The basic syntax of the **CREATE TABLE** statement is:

```
CREATE TABLE TableName (
    ColumnName1 DataType Constraints,
    ColumnName2 DataType Constraints,
    ...
);
```

- **TableName**: Name of the table.
- **ColumnName**: Name of a column (attribute).
- **DataType**: Defines the type of data a column can store (e.g., `VARCHAR`, `INTEGER`).
- **Constraints**: Optional rules like `PRIMARY KEY`, `NOT NULL`.

---

2. Example 1: Creating a Simple Table

Let's create a **Provinces** table in Canada:

```
CREATE TABLE Provinces (
    id CHAR(2) PRIMARY KEY NOT NULL,
    name VARCHAR(24) NOT NULL
);
```

📌 **Breakdown**:

- `id CHAR(2)`: Stores **province abbreviations** (e.g., 'AB', 'BC').
- `name VARCHAR(24)`: Stores **full province names** (e.g., "Alberta", "British Columbia").
- `PRIMARY KEY`: Ensures `id` is **unique** and cannot be duplicated.

- `NOT NULL`: Ensures values **must be provided**.

---

3. Example 2: Creating an AUTHOR Table

Using the **Library Database** example, let's create an **AUTHOR** table:

```
CREATE TABLE Author (
    author_id CHAR(2) PRIMARY KEY NOT NULL,
    lastname VARCHAR(15) NOT NULL,
    firstname VARCHAR(15) NOT NULL,
    email VARCHAR(40),
    city VARCHAR(15),
    country CHAR(2)
);
```

📌 **Key Details**:

- `author_id`: **Primary Key** (unique identifier for authors).
- `lastname`, `firstname`: Cannot be `NULL` (every author must have a name).
- `email`, `city`, `country`: Optional fields.

---

4. Key Takeaways

✓ **CREATE** is a **DDL statement** used to create **tables**.
✓ Defines **columns, data types, and constraints**.
✓ A **Primary Key (PK)** uniquely identifies each row.
✓ `NOT NULL` ensures **required fields** cannot be empty.

ALTER, DROP, and Truncate Tables

SQL provides **three powerful commands** to modify, remove, or clear data from tables:
1- **ALTER TABLE** – Modify the structure of an existing table.
2- **DROP TABLE** – Completely delete a table.
3- **TRUNCATE TABLE** – Remove all data from a table while keeping its structure.

---

1. ALTER TABLE 🛠

Used to **add, modify, or remove columns, keys, or constraints** in an existing table.

Syntax:

```
ALTER TABLE table_name ADD column_name data_type;
ALTER TABLE table_name MODIFY column_name new_data_type;
ALTER TABLE table_name DROP COLUMN column_name;
```

Example 1: Adding a Column

Add a **telephone number** column to the `author` table:

```
ALTER TABLE author ADD COLUMN telephone_number BIGINT;
```

- `BIGINT` is used to store numbers up to **19 digits** long.

Example 2: Modifying a Column

Change `telephone_number` to **CHAR(20)** so it can store **symbols like +, -, ()**:

```
ALTER TABLE author MODIFY telephone_number CHAR(20);
```

📌 **Note:** Changing data types may fail if existing data is **incompatible** with the new type.

Example 3: Removing a Column

Remove the `telephone_number` column if it is no longer needed:

```
ALTER TABLE author DROP COLUMN telephone_number;
```

---

2. DROP TABLE ❌

Used to **delete an entire table** along with all its data.

Syntax:

```
DROP TABLE table_name;
```

Example: Deleting the `author` Table

```
DROP TABLE author;
```

⚠ **Warning:** This permanently deletes the table and its data!

---

3. TRUNCATE TABLE 🚀

Used to **quickly remove all rows from a table** while keeping the table structure intact.

Syntax:

```
TRUNCATE TABLE table_name IMMEDIATE;
```

Example: Clearing the `author` Table

```
TRUNCATE TABLE author IMMEDIATE;
```

⚡ **Faster than DELETE** because it does not log individual row deletions.
⏳ **CANNOT be undone** because it is processed **immediately**.

---

4. Key Takeaways

✓ `ALTER TABLE` modifies a table's **structure** (add, modify, remove columns).
✓ `DROP TABLE` **completely removes** a table and its data.
✓ `TRUNCATE TABLE` **deletes all rows** but **keeps the table**.

Summary: Relational Database Concepts and Tables

- A database is a repository of data that provides functionality for adding, modifying, and querying the data.
- SQL is a language used to query or retrieve data from a relational database.
- The Relational Model is the most used data model for databases because it allows for data independence.
- The primary key of a relational table uniquely identifies each tuple or row, preventing duplication of data and providing a way of defining relationships between tables.
- SQL statements fall into two different categories: Data Definition Language (DDL) statements and Data Manipulation Language (DML) statements.

Module three

Refining your Results

Using String Patterns and Ranges

In SQL, **retrieving data efficiently** is crucial. You can **simplify SELECT statements** using:
1- **String patterns (`LIKE` & wildcards)**
2- **Ranges (`BETWEEN` operator)**
3- **Sets of values (`IN` operator)**

---

1. Using String Patterns with `LIKE` 🔎

The `LIKE` operator helps when you **don't know the exact value** but remember part of it.

Syntax:

```
SELECT column_name FROM table_name WHERE column_name LIKE 'pattern%';
```

- `%` **(percent sign)** = Matches **any sequence of characters**
- `_` **(underscore)** = Matches **one** character

Example: Search for Authors Whose First Name Starts with "R"

```
SELECT firstname FROM author WHERE firstname LIKE 'R%';
```

- **Returns:** *Raul, Rav* (any name starting with "R")

More Examples:

```
SELECT title FROM book WHERE title LIKE '%Story';
-- Returns books where the title ends with "Story"

SELECT email FROM users WHERE email LIKE '_@gmail.com';
-- Returns emails that have exactly one character before @gmail.com
```

---

2. Using Ranges with `BETWEEN` 🔢

Instead of writing **multiple conditions**, use `BETWEEN` for numeric or date ranges.

Syntax:

```
SELECT column_name FROM table_name WHERE column_name BETWEEN value1
AND value2;
```

📌 **Both values are included** in the range.

Example: Books with Pages Between 290 and 300

```
SELECT title FROM book WHERE pages BETWEEN 290 AND 300;
```

🔹 **Same as:**

```
SELECT title FROM book WHERE pages >= 290 AND pages <= 300;
```

✅ **BETWEEN is simpler and more readable!**

---

3. Using Sets with `IN` 🎯

The `IN` operator lets you **match multiple values without using multiple OR conditions**.

Syntax:

```
SELECT column_name FROM table_name WHERE column_name IN (value1,
value2, value3);
```

Example: Find Authors from Specific Countries

```
SELECT firstname, lastname FROM author WHERE country IN ('Canada',
'India', 'China');
```

🔹 **Same as:**

```
SELECT firstname, lastname FROM author
WHERE country = 'Canada' OR country = 'India' OR country = 'China';
```

✅ **`IN` makes queries cleaner and shorter!**

---

4. Key Takeaways

✓ Use `LIKE` with `%` or _ for **partial string matching**.
✓ Use `BETWEEN` for **numeric or date ranges**.
✓ Use `IN` for **multiple specific values**.

Sorting Result Sets

1. Sorting in Ascending Order (Default)

To sort data alphabetically or numerically **in increasing order**, use `ORDER BY`.

Syntax:

```
SELECT column_name FROM table_name ORDER BY column_name;
```

- **Default sort order is ASC (ascending).**

Example: Sort Books Alphabetically by Title

```
SELECT title FROM book ORDER BY title;
```

✅ **Books are listed from A → Z.**

---

2. Sorting in Descending Order (`DESC`)

To sort **in decreasing order**, add `DESC`.

Example: Sort Books in Reverse Alphabetical Order

```
SELECT title FROM book ORDER BY title DESC;
```

✅ **Books are listed from Z → A.**

---

3. Sorting by Numeric Values

You can sort by **numeric** values such as **number of pages, prices, or IDs**.

Example: Sort Books by Number of Pages (Lowest to Highest)

```
SELECT title, pages FROM book ORDER BY pages;
```

✅ **Books with fewer pages appear first.**

Example: Sort Books by Number of Pages (Highest to Lowest)

```
SELECT title, pages FROM book ORDER BY pages DESC;
```

✅ **Books with more pages appear first.**

---

4. Sorting by Column Position (Using Column Index)

Instead of specifying the column name, you can use its **position** in the `SELECT` statement.

Example: Sort Books by the 2nd Column (`pages`)

```
SELECT title, pages FROM book ORDER BY 2;
```

✅ **Equivalent to:**

```
SELECT title, pages FROM book ORDER BY pages;
```

📌 **Column index is useful for dynamic queries but not recommended for clarity.**

---

5. Sorting by Multiple Columns

You can sort by **multiple columns** for more precise ordering.

Example: Sort by Title (A-Z) and then by Pages (Fewest to Most)

```
SELECT title, pages FROM book ORDER BY title, pages;
```

✅ **Books with the same title are sorted by pages.**

Example: Sort by Pages (Most to Fewest) and then by Title (Z-A)

```
SELECT title, pages FROM book ORDER BY pages DESC, title DESC;
```

✅ **Sorting by multiple columns ensures a more structured result!**

---

6. Key Takeaways

✓ **`ORDER BY` sorts results in ascending (`ASC`) or descending (`DESC`) order.**
✓ **Sorting works with text, numbers, and dates.**
✓ **You can use column names or their positions for sorting.**
✓ **Multiple column sorting provides refined results.**

Grouping Result Sets

Grouping and eliminating duplicates from a result set helps organize and analyze your data more efficiently. By using the `DISTINCT`, `GROUP BY`, and `HAVING` clauses, you can refine your queries to display only the data you need.

---

1. Eliminate Duplicates with `DISTINCT`

When you run a query that retrieves data from a column, you may encounter **duplicate values**. To eliminate duplicates, use the `DISTINCT` keyword.

Example: List of Authors' Countries

```
SELECT DISTINCT country FROM author ORDER BY 1;
```

✅ **Result:** Only unique country codes will be listed.

---

2. Grouping Data with `GROUP BY`

If you want to know how many authors belong to each country, you can **group** the data using the `GROUP BY` clause. This groups the result set based on one or more columns.

Example: Count of Authors per Country

```
SELECT country, COUNT(*) FROM author GROUP BY country;
```

✅ **Result:** The result will display each country and the number of authors from that country.

---

3. Rename Calculated Columns Using `AS`

By default, when using aggregation functions like `COUNT()`, the result column may have a default name (like `COUNT(*)`). To give the calculated column a more meaningful name, use the `AS` keyword.

Example: Rename the Count Column

```
SELECT country, COUNT(*) AS author_count FROM author GROUP BY
country;
```

✅ **Result:** The second column will be labeled `author_count` instead of just `COUNT(*)`.

---

4. Restrict Results Using `HAVING`

The `HAVING` clause is used to filter grouped data based on conditions. It works only with **grouped** data and is used in combination with `GROUP BY`.

Example: Authors per Country Greater than 4

```
SELECT country, COUNT(*) AS author_count
FROM author
GROUP BY country
HAVING COUNT(*) > 4;
```

✅ **Result:** Only countries with more than 4 authors will be displayed in the result set.

---

5. Key Differences Between `WHERE` and `HAVING`

- **`WHERE` clause**: Filters rows **before** grouping. It's applied to individual rows.
- **`HAVING` clause**: Filters groups **after** the grouping is done. It’s applied to aggregate values.

---

6. Key Takeaways

- **Use `DISTINCT`** to eliminate duplicate values in a result set.
- **Use `GROUP BY`** to group data and perform aggregate functions like `COUNT()`.
- **Use `HAVING`** to filter results after grouping, based on aggregate values.

Lesson’s Summary

- You can use the WHERE clause to refine your query results.
- The search condition of the WHERE clause uses a predicate to refine the search.
- You can use the wildcard character (%) as a substitute for unknown characters in a pattern.
- You can use BETWEEN ... AND ... to specify a range of numbers.
- You can sort query results into ascending or descending order, using the ORDER BY clause to specify the column to sort on.
- You can group query results by using the GROUP BY clause.

Functions, Multiple Tables, and Sub-queries

Built-in Database Functions

SQL provides **built-in functions** that allow you to perform operations on data **within the database** instead of retrieving the data and processing it in an application. This improves **efficiency**, reduces **network traffic**, and enhances **performance** when dealing with large datasets.

---

1. Aggregate Functions

Aggregate functions operate on a **set of values** and return a **single value**.

Common Aggregate Functions:

Function	Description
`SUM(column)`	Returns the total sum of values in a column.
`MIN(column)`	Returns the smallest value in a column.
`MAX(column)`	Returns the largest value in a column.
`AVG(column)`	Returns the average (mean) of the values in a column.
`COUNT(column)`	Returns the number of rows with a non-null value in a column.

Example Queries

1. Total Cost of Pet Rescues

```
SELECT SUM(COST) AS SUM_OF_COST FROM PETRESCUE;
```

2. Maximum Quantity in a Single Transaction

```
SELECT MAX(QUANTITY) FROM PETRESCUE;
```

3. Minimum ID for Rescued Dogs

```
SELECT MIN(ID) FROM PETRESCUE WHERE ANIMAL = 'Dog';
```

4. Average Cost of Rescues

```
SELECT AVG(COST) FROM PETRESCUE;
```

5. Average Cost Per Dog

```
SELECT AVG(COST / QUANTITY) FROM PETRESCUE WHERE ANIMAL = 'Dog';
```

📌 *This query first divides `COST` by `QUANTITY` (to get the cost per animal) before calculating the average.*

---

2. Scalar Functions

Scalar functions **operate on individual values** rather than a set of values.

Common Scalar Functions:

Function	Description
`ROUND(value, decimal_places)`	Rounds a number to a specified decimal place.
`LENGTH(string)`	Returns the length (number of characters) of a string.
`UPPER(string)`	Converts a string to **uppercase**.
`LOWER(string)`	Converts a string to **lowercase**.

Example Queries

1. Round Cost to the Nearest Integer

```
SELECT ROUND(COST) FROM PETRESCUE;
```

2. Length of Each Animal Name

```
SELECT LENGTH(ANIMAL) FROM PETRESCUE;
```

3. Convert Animal Names to Uppercase

```
SELECT UPPER(ANIMAL) FROM PETRESCUE;
```

4. Convert Animal Names to Lowercase in a `WHERE` Clause

```
SELECT * FROM PETRESCUE WHERE LOWER(ANIMAL) = 'cat';
```

📌 *This ensures that even if data is stored in mixed case, the query matches it correctly.*

5. Retrieve Unique Animal Names in Uppercase

```
SELECT DISTINCT UPPER(ANIMAL) FROM PETRESCUE;
```

📌 *This applies the `UPPER()` function to each distinct value in the `ANIMAL` column.*

---

3. Combining Functions

SQL functions can be **nested** inside each other for more advanced queries.

Example: Count Distinct Animals in Uppercase

```
SELECT COUNT(DISTINCT UPPER(ANIMAL)) FROM PETRESCUE;
```

📌 *This counts the number of unique animals in uppercase format.*

Key Takeaways

✅ **Aggregate Functions** operate on multiple rows and return a **single** result (e.g., `SUM`, `AVG`, `MIN`, `MAX`).
✅ **Scalar Functions** operate on individual values (e.g., `ROUND`, `UPPER`, `LOWER`).
✅ **String Functions** manipulate text data (e.g., `LENGTH`, `UPPER`, `LOWER`).
✅ **Combining Functions** allows for **powerful** data transformations.

Date and Time Built-in Functions

SQL provides **date and time functions** to manipulate **dates, times, and timestamps** stored in a database. These functions help with **extracting, formatting, and performing calculations** on date and time values.

---

1. Date and Time Data Types

Most SQL databases provide the following date/time data types:

Data Type	Format	Example Value	Description
`DATE`	`YYYYMMDD`	`20250304`	Stores only the **date**.
`TIME`	`HHMMSS`	`153045`	Stores only the **time**.
`TIMESTAMP`	`YYYYMMDDHHMMSSZZZZZZ`	`20250304153045987654`	Stores **both date and time**, including microseconds.

---

2. Extracting Date and Time Components

SQL provides functions to extract specific parts of a date/time value.

Function	Description	Example Query
`DAY(date_column)`	Extracts the **day** from a date.	`SELECT DAY(rescue_date) FROM PETRESCUE;`

Function	Description	Example Query
`MONTH(date_column)`	Extracts the **month** from a date.	`SELECT MONTH(rescue_date) FROM PETRESCUE;`
`YEAR(date_column)`	Extracts the **year** from a date.	`SELECT YEAR(rescue_date) FROM PETRESCUE;`
`DAYOFMONTH(date_column)`	Extracts the **day of the month**.	`SELECT DAYOFMONTH(rescue_date) FROM PETRESCUE;`
`DAYOFWEEK(date_column)`	Extracts the **day of the week** (1 = Sunday, 7 = Saturday).	`SELECT DAYOFWEEK(rescue_date) FROM PETRESCUE;`
`DAYOFYEAR(date_column)`	Extracts the **day of the year** (1-366).	`SELECT DAYOFYEAR(rescue_date) FROM PETRESCUE;`
`WEEK(date_column)`	Extracts the **week number** (1-52).	`SELECT WEEK(rescue_date) FROM PETRESCUE;`
`HOUR(time_column)`	Extracts the **hour** from a time value.	`SELECT HOUR(rescue_time) FROM PETRESCUE;`
`MINUTE(time_column)`	Extracts the **minute** from a time value.	`SELECT MINUTE(rescue_time) FROM PETRESCUE;`
`SECOND(time_column)`	Extracts the **second** from a time value.	`SELECT SECOND(rescue_time) FROM PETRESCUE;`

---

3. Date and Time Functions in the WHERE Clause

Date/time functions can be used to filter data.

Example 1: Get Rescues That Happened in May (Month = 5)

```
SELECT COUNT(*) FROM PETRESCUE
WHERE MONTH(rescue_date) = 5;
```

Example 2: Get Rescues That Happened on a Sunday

```
SELECT * FROM PETRESCUE
WHERE DAYOFWEEK(rescue_date) = 1;
```

📌 *`DAYOFWEEK(rescue_date) = 1` means **Sunday**.*

---

4. Performing Date and Time Arithmetic

SQL allows **date and time calculations** using functions like `DATE_ADD` and `DATEDIFF`.

Example 1: Find the Date Three Days After Each Rescue

```
SELECT DATE_ADD(rescue_date, INTERVAL 3 DAY)
FROM PETRESCUE;
```

📌 *This adds 3 days to each* `rescue_date`.

Example 2: Find How Many Days Have Passed Since Each Rescue

```
SELECT DATEDIFF(CURRENT_DATE, rescue_date)
FROM PETRESCUE;
```

📌 *This calculates the number of days between* `rescue_date` *and today.*

---

5. Using Special Registers (`CURRENT_DATE`, `CURRENT_TIME`)

SQL provides special registers to get the **current date and time**.

Register	Description	Example Query
`CURRENT_DATE`	Returns today's date.	`SELECT CURRENT_DATE;`
`CURRENT_TIME`	Returns the current time.	`SELECT CURRENT_TIME;`
`CURRENT_TIMESTAMP`	Returns the current date and time.	`SELECT CURRENT_TIMESTAMP;`

Key Takeaways

✅ **Date and Time Data Types**: `DATE`, `TIME`, `TIMESTAMP`.
✅ **Extracting Date Components**: Use `DAY()`, `MONTH()`, `YEAR()`, etc.
✅ **Filtering Data by Date**: Use functions like `MONTH()`, `DAYOFWEEK()` in `WHERE` clauses.
✅ **Date Arithmetic**: Use `DATE_ADD()` and `DATEDIFF()` to perform calculations.
✅ **Special Registers**: Use `CURRENT_DATE` and `CURRENT_TIME` for real-time queries.

Sub-Queries and Nested Selects

A **subquery** (or **nested query**) is a query **inside another query**. Subqueries allow you to create **more complex and powerful SQL queries** by using the result of one query within another.

---

1. What is a Subquery?

A subquery is enclosed in **parentheses** and can be used in different parts of a SQL query, including: ✅ **WHERE Clause**
✅ **SELECT Clause** (Column Expressions)
✅ **FROM Clause** (Derived Tables)

---

2. Subqueries in the WHERE Clause

Subqueries are commonly used in the `WHERE` clause to filter results **based on another query's output**.

Example: Employees Who Earn More Than the Average Salary

❌ *This query will result in an error:*

```
SELECT * FROM employees
WHERE salary > AVG(salary);
```

📌 *The aggregate function `AVG(salary)` **cannot be used directly** in the `WHERE` clause.*

✅ **Correct Approach: Using a Subquery**

```
SELECT employee_id, first_name, last_name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

📌 *Here, the subquery `(SELECT AVG(salary) FROM employees)` calculates the average salary first, allowing the outer query to compare each salary with it.*

---

3. Subqueries in the SELECT Clause (Column Expressions)

A subquery can be used in the **SELECT statement** to add **calculated values** as columns.

Example: Show Each Employee's Salary Along with the Average Salary

❌ *This query will result in an error:*

```
SELECT employee_id, salary, AVG(salary) AS avg_salary
FROM employees;
```

📌 *This fails because an aggregate function **requires a GROUP BY clause**.*

✅ **Correct Approach: Using a Subquery in the SELECT Clause**

```
SELECT employee_id, salary,
       (SELECT AVG(salary) FROM employees) AS avg_salary
FROM employees;
```

📌 *Here, the subquery calculates the **same average salary** for every row.*

---

4. Subqueries in the FROM Clause (Derived Tables)

A subquery in the `FROM` clause is called a **Derived Table** (or **Table Expression**). It acts as a **temporary table** for the outer query.

Example: Create a Table Without Sensitive Information

```
SELECT *
FROM (SELECT employee_id, first_name, last_name, department_id
      FROM employees) AS employee_info;
```

📌 *This subquery hides sensitive columns (like `salary`, `date_of_birth`) before selecting data.*

---

5. Key Takeaways

✅ **Subqueries in the WHERE Clause** → Used for filtering results.
✅ **Subqueries in the SELECT Clause** → Used for calculated columns.
✅ **Subqueries in the FROM Clause** → Used as temporary tables.

Working with Multiple Tables

SQL provides **multiple ways** to access data from **more than one table** in a query. The two common methods covered here are:
✅ **Subqueries**
✅ **Implicit Joins**

---

1. Using Subqueries to Access Multiple Tables

A **subquery** (nested query) is used to **filter results** based on data from another table.

Example 1: Retrieve Employees with a Department ID Present in the Departments Table

```
SELECT * FROM employees
WHERE department_id IN (SELECT department_id FROM departments);
```

📌 *Here, the subquery gets the list of* `department_id` *values from the* `departments` *table, and the outer query selects employees whose* `department_id` *exists in that list.*

Example 2: Retrieve Employees from a Specific Location

If we want to find employees working in **Location L0002**, we can use a **subquery** on the `departments` table:

```
SELECT * FROM employees
WHERE department_id IN
    (SELECT department_id FROM departments WHERE location_id =
'L0002');
```

📌 *This first retrieves department IDs for* `location_id = 'L0002'` *and then selects employees from those departments.*

Example 3: Get Department Details for Employees Earning More Than $70,000

We need to first find department IDs of employees **earning more than 70,000**, then retrieve their department names.

```
SELECT department_id, department_name
FROM departments
WHERE department_id IN
    (SELECT department_id FROM employees WHERE salary > 70000);
```

📌 *The inner query selects department IDs from employees earning above $70K, and the outer query retrieves department details for those IDs.*

---

2. Using Implicit Joins (Comma-Separated Table List)

Instead of using subqueries, we can **list multiple tables** in the `FROM` clause.

Example 4: Join Employees and Departments (Cross Join)

```
SELECT * FROM employees, departments;
```

📌 *This generates a **full join** (cross join), where **every row** from `employees` is combined with **every row** from `departments` → This is usually not useful and results in a huge number of rows.*

Example 5: Join Employees and Departments with Matching Department IDs

To get **only valid employee-department matches**, we use a **WHERE clause**:

```
SELECT *
FROM employees, departments
WHERE employees.department_id = departments.department_id;
```

📌 *This filters results to only include employees who belong to a valid department.*

Example 6: Using Table Aliases for Simplicity

Instead of repeating long table names, we use **aliases** (`E` for employees, `D` for departments):

```
SELECT *
FROM employees E, departments D
WHERE E.department_id = D.department_id;
```

📌 *Aliases make queries **cleaner** and **easier to read**.*

Example 7: Get Employee IDs Along with Department Names

```
SELECT E.employee_id, D.department_name
FROM employees E, departments D
WHERE E.department_id = D.department_id;
```

📌 *Now, instead of showing all data, we **specifically select** employee IDs and their department names.*

Key Takeaways

✅ **Subqueries** → Used when filtering data using another table.
✅ **Implicit Joins** → Used when selecting data from multiple tables in one query.
✅ **Table Aliases** → Make queries **shorter and more readable**.

Lesson's Summary

- Tools for database management that offer built-in functions for performing operations on data within the database itself.
- That when working with large datasets, you may save time by using built-in functions rather than first retrieving the data into your application and then executing functions on the retrieved data.
- You can use sub-queries to form more powerful queries than otherwise.
- You can use a sub-select expression to evaluate some built-in aggregate functions like the average function.
- Derived tables or table expressions are sub-queries where the outer query uses the results of the sub-query as a data source.

Module four

Accessing databases using Python

How to Access Databases Using Python

1. Why Use Python for Databases?

Python is **widely used** in data science because:
🔹 It has **rich libraries** like **NumPy, Pandas, Matplotlib, and SciPy**.
🔹 It is **easy to learn** with a **simple syntax**.
🔹 It is **cross-platform**, meaning Python programs run on multiple operating systems **without modification**.
🔹 It has a **Database API (DB API)** that makes database access **easier**.

---

2. Jupyter Notebooks for Database Access

Jupyter Notebooks are **widely used** in data science because:
📌 They support **40+ languages** (Python, R, Julia, Scala).
📌 They allow **sharing** via **GitHub, Dropbox, or email**.
📌 They support **interactive outputs** like **HTML, images, LaTeX, and videos**.
📌 They integrate with **big data tools** like **Apache Spark, Pandas, TensorFlow, and Scikit-learn**.

---

3. Connecting Python to Databases

Python communicates with **DBMS (Database Management Systems)** using **APIs (Application Programming Interfaces)**.

- The **SQL API** allows Python to send SQL queries to the **DBMS** and retrieve results.
- The program connects to the database via **API calls**, sends SQL commands, retrieves results, and finally **disconnects**.

---

4. Proprietary SQL APIs for Popular Databases

DBMS	Python API	Purpose
MySQL	`MySQL C API`	Low-level MySQL access
PostgreSQL	`psycopg2`	Python → PostgreSQL
IBM DB2	`IBM_DB API`	Python → IBM DB2
SQL Server	`dblib`	Python → SQL Server
Microsoft Databases	`ODBC`	Windows Database Access
Oracle	`OCI`	Python → Oracle
Java-based Apps	`JDBC`	Java → Databases

---

5. Python Code to Connect to a Database

Example: Connecting Python to SQLite Database

```
import sqlite3

# Connect to the database (or create if not exists)
conn = sqlite3.connect("my_database.db")

# Create a cursor object to execute SQL commands
cursor = conn.cursor()

# Create a table
cursor.execute("""
    CREATE TABLE IF NOT EXISTS employees (
        id INTEGER PRIMARY KEY,
        name TEXT,
        salary REAL
    )
""")

# Insert data
cursor.execute("INSERT INTO employees (name, salary) VALUES (?, ?)",
("John Doe", 70000))
```

```
# Commit changes and close connection
conn.commit()
conn.close()
```

📌 This code **creates a database**, defines a **table**, and **inserts data** into it.

---

6. Querying Data from the Database

Example: Fetch Data Using Python

```
import sqlite3

# Connect to database
conn = sqlite3.connect("my_database.db")
cursor = conn.cursor()

# Fetch all employees
cursor.execute("SELECT * FROM employees")
rows = cursor.fetchall()

# Print results
for row in rows:
    print(row)

# Close connection
conn.close()
```

📌 This **retrieves all employee records** and **prints them**.

Writing code using DB-API

1. What is DB-API?

DB-API is the **Python standard API** for relational databases. It allows Python programs to interact with various **DBMS (Database Management Systems) without modifying the code**.

Advantages of DB-API

- **Easy to use** and **understand**
- **Consistent** across different databases
- **Portable** – code works with multiple databases
- **Broad compatibility** with various DBMS systems

---

2. Python Libraries for Database Connections

Each database system has its own **Python DB-API library**:

Database	Python Library
IBM DB2	`IBM_DB`
MySQL	`mysql-connector-python`
PostgreSQL	`psycopg2`
MongoDB	`PyMongo`

---

3. DB-API Objects: Connections and Cursors

- 🔹 **Connection Objects** → Connect to a **database** and manage **transactions**.
- 🔹 **Cursor Objects** → Run **queries** and **fetch** results.

DB-API Connection Methods

Method	Function
`cursor()`	Returns a new cursor object
`commit()`	Saves changes to the database
`rollback()`	Rolls back uncommitted changes
`close()`	Closes the database connection

---

4. How a Cursor Works

A **cursor** is like a **file handle**:
📌 **Opens** a query result → **Iterates** over records → **Fetches** data → **Closes** after use

Cursors created from the **same connection**:
✓ **Share visibility** of database changes

Cursors from **different connections**:
✓ **May or may not** be isolated, depending on **transaction support**

---

5. Writing Python Code to Access a Database

Example: Connecting Python to a MySQL Database

```
import mysql.connector

# Connect to MySQL database
conn = mysql.connector.connect(
    host="localhost",
```

```
    user="root",
    password="password",
    database="company_db"
)

# Create a cursor object
cursor = conn.cursor()

# Create a table
cursor.execute("""
    CREATE TABLE IF NOT EXISTS employees (
        id INT AUTO_INCREMENT PRIMARY KEY,
        name VARCHAR(100),
        salary FLOAT
    )
""")

# Insert data
cursor.execute("INSERT INTO employees (name, salary) VALUES (%s,
%s)", ("Alice", 60000))
cursor.execute("INSERT INTO employees (name, salary) VALUES (%s,
%s)", ("Bob", 70000))

# Commit changes
conn.commit()

# Close the connection
conn.close()
```

📌 **Steps Explained:**
✓ Establishes a **connection** to MySQL
✓ Creates a **cursor** to execute SQL
✓ Executes **SQL commands** to create a table and insert data
✓ **Commits** changes and **closes** the connection

---

6. Querying Data from the Database

```
import mysql.connector

# Connect to MySQL
conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password="password",
    database="company_db"
)

# Create a cursor
cursor = conn.cursor()

# Execute a SELECT query
cursor.execute("SELECT * FROM employees")

# Fetch all results
rows = cursor.fetchall()
```

```
# Print results
for row in rows:
    print(row)

# Close connection
conn.close()
```

📌 **Steps Explained:**
✓ **Fetches** all employee records
✓ **Prints** query results
✓ **Closes** the connection

---

7. Best Practices for DB-API

✅ Always **close connections** to free resources
✅ Use **parameterized queries** to prevent SQL injection
✅ **Commit** changes to **save transactions**
✅ Handle **errors** using **try-except** blocks

---

8. Summary

🚀 With **DB-API**, you can **connect to databases**, **execute queries**, and **fetch results** efficiently!

Accessing Databases with SQL Magic

SQL Magic in Jupyter Notebooks

After this guide, you will be able to:
✅ **Describe** Magic Statements in Jupyter Notebooks
✅ **Differentiate** between Line Magics and Cell Magics
✅ **Use** popular magic commands
✅ **Apply** SQL Magic to query databases in Jupyter

---

1. What are Magic Statements?

Magic commands (or magic functions) in Jupyter Notebooks provide special functionalities that affect the behavior of the notebook.

- **They are not standard Python code**
- **Help solve common data analysis tasks**

---

2. Types of Magic Statements

There are **two types** of magic commands in Jupyter:

Type	Prefix	Scope	Example
Line Magic	`%`	Single Line	`%pwd`
Cell Magic	`%%`	Whole Cell	`%%timeit`

- **Line magics** work on **one line** at a time
- **Cell magics** work on **multiple lines (entire cell)**

---

3. Commonly Used Magic Commands

🔹 Line Magics

Command	Function
`%pwd`	Prints current working directory
`%ls`	Lists files in the current directory
`%history`	Displays command history
`%reset`	Clears all variables
`%who`	Lists all variables in the namespace
`%whos`	Lists all variables with details
`%matplotlib inline`	Displays matplotlib plots inside the notebook
`%timeit`	Times execution of a single statement

👉 To list all available **Line Magics**, run:

```
%lsmagic
```

📌 **Using multiple line magics in one cell**

```
%pwd
%ls
```

Each line magic **must be on a separate line**.

🔹 Cell Magics

Command	Function
`%%timeit`	Times execution of the entire cell
`%%writefile filename.txt`	Writes cell content to a file
`%%HTML`	Runs HTML code inside a cell
`%%javascript` or `%%js`	Runs JavaScript code inside a cell
`%%bash`	Runs Bash shell commands inside a cell

📌 Example: Writing HTML in Jupyter

```
%%HTML
<h1>Hello, World!</h1>
```

📌 Example: Running JavaScript in Jupyter

```
%%javascript
alert("Hello, World!");
```

📌 Example: Running Bash in Jupyter

```
%%bash
echo "Hello, World!"
```

---

4. Using SQL Magic in Jupyter Notebooks

📌 Step 1: Install Dependencies

Run the following command in a **Jupyter Notebook cell** to install the `ipython-sql` package:

```
!pip install ipython-sql
```

📌 Step 2: Load SQL Extension

```
%load_ext sql
```

📌 Step 3: Establish a Database Connection

For example, to connect to a **SQLite** database (`HR.db`):

```
%sql sqlite:///HR.db
```

📌 Step 4: Running SQL Queries

🔹 Using Line Magic (`%sql`)

```
%sql SELECT * FROM employees;
```

🔹 Using Cell Magic (`%%sql`)

```
%%sql
```

```
SELECT * FROM employees;
```

📌 Step 5: Storing Query Results into Variables

```
result = %sql SELECT name, salary FROM employees;
print(result)
```

Analyzing Data with Python

After this guide, you will be able to:
✅ **Load** and explore datasets using Python
✅ **Use** SQLite to store and query data
✅ **Perform** Exploratory Data Analysis (EDA) with Pandas
✅ **Visualize** data using Seaborn

---

1. Loading Data into SQLite

📌 Step 1: Import Dependencies

```
import pandas as pd
import sqlite3
```

📌 Step 2: Load the Dataset

The dataset, containing **McDonald's menu nutrition facts**, is loaded into a Pandas DataFrame:

```
df = pd.read_csv("mcdonalds_nutrition.csv")
```

📌 Step 3: Store Data in SQLite

```
conn = sqlite3.connect("mcdonalds.db")
df.to_sql("MCDONALDS_NUTRITION", conn, if_exists="replace",
index=False)
```

📌 Step 4: Query the Database

```
query = "SELECT * FROM MCDONALDS_NUTRITION LIMIT 5"
df_query = pd.read_sql(query, conn)
```

---

2. Performing Exploratory Data Analysis (EDA)

📌 Step 1: Summary Statistics

```
df.describe()
```

🔹 The dataset contains **260 observations** and **9 unique food categories**.

📌 **Step 2: Identifying High Sodium Foods**

```
df.loc[df["Sodium"].idxmax(), ["Item", "Sodium"]]
```

🔹 The **highest sodium content** is **3600 mg**, found in **Chicken McNuggets (40 pieces)**.

---

3. Visualizing Data with Seaborn

Scatter Plot (Protein vs. Fat)

```
import seaborn as sns
sns.jointplot(x="Protein", y="Total Fat", data=df, kind="scatter")
```

🔹 The correlation coefficient is **0.81**, indicating a strong **positive correlation** between protein and fat.

Box Plot (Sugar Content Distribution)

```
sns.boxplot(x=df["Sugars"])
```

🔹 The **median sugar content** is around **30g**, with some **outliers reaching 128g**.

---

4. Summary

🚀 Now you can:
✓ **Load and store** datasets in SQLite
✓ **Query** databases using SQL in Python
✓ **Perform** basic data analysis with Pandas
✓ **Visualize** distributions and correlations using Seaborn

🔹 These techniques are essential for **data analysis and insights generation**!

Analyzing Data with Python

After this guide, you will be able to:
✅ **Load** and explore datasets using Python
✅ **Use** SQLite to store and query data

✅ **Perform** Exploratory Data Analysis (EDA) with Pandas
✅ **Visualize** data using Seaborn

---

1. Loading Data into SQLite

📌 Step 1: Import Dependencies

```
import pandas as pd
import sqlite3
```

📌 Step 2: Load the Dataset

The dataset, containing **McDonald's menu nutrition facts**, is loaded into a Pandas DataFrame:

```
df = pd.read_csv("mcdonalds_nutrition.csv")
```

📌 Step 3: Store Data in SQLite

```
conn = sqlite3.connect("mcdonalds.db")
df.to_sql("MCDONALDS_NUTRITION", conn, if_exists="replace",
index=False)
```

📌 Step 4: Query the Database

```
query = "SELECT * FROM MCDONALDS_NUTRITION LIMIT 5"
df_query = pd.read_sql(query, conn)
```

---

2. Performing Exploratory Data Analysis (EDA)

📌 Step 1: Summary Statistics

```
df.describe()
```

🔹 The dataset contains **260 observations** and **9 unique food categories**.

📌 Step 2: Identifying High Sodium Foods

```
df.loc[df["Sodium"].idxmax(), ["Item", "Sodium"]]
```

🔹 The **highest sodium content** is **3600 mg**, found in **Chicken McNuggets (40 pieces)**.

---

3. Visualizing Data with Seaborn

Scatter Plot (Protein vs. Fat)

```
import seaborn as sns
sns.jointplot(x="Protein", y="Total Fat", data=df, kind="scatter")
```

🔹 The correlation coefficient is **0.81**, indicating a strong **positive correlation** between protein and fat.

Box Plot (Sugar Content Distribution)

```
sns.boxplot(x=df["Sugars"])
```

🔹 The **median sugar content** is around **30g**, with some **outliers reaching 128g**.

---

4. Summary

🚀 Now you can:
✓ **Load and store** datasets in SQLite
✓ **Query** databases using SQL in Python
✓ **Perform** basic data analysis with Pandas
✓ **Visualize** distributions and correlations using Seaborn

Lesson's Summary

- Magic commands are special commands that provide special functionalities.
- Cell magics are commands prefixed with two %% characters and operate on multiple input lines.
- DB APIs are commands prefixed with two %% characters and operate on multiple input lines.
- The two main concepts in the Python DB API are Connection Objects and Query Objects.
- A database cursor is a control structure that enables traversal over the records in a database.
- Pandas' methods are equipped with common mathematical and statistical methods.
- The pandas.read_csv() function is used to read the database CSV file.
- The sqlite3.connect() function is used to connect to a database.
- To use pandas to retrieve data from the database tables, load data using the read_sql method and select the SQL Select Query
- A categorical scatterplot is created using the swarmplot() method by the seaborn package.

Module five

Assignment Preparation: Working with real-world data sets and built-in SQL functions

Working with Real World Datasets

At the end of this guide, you will be able to:
✅ **Discuss** the format of data in databases
✅ **Explain** how to load data into databases

---

1. Understanding CSV Files

- Many real-world datasets are available as **CSV (Comma-Separated Values) files**
- CSV files contain **text-based data** where values are typically separated by **commas** (`","`), though other separators like semicolons (`";"`) may also be used
- The **first row** in a CSV file usually contains **attribute labels** (column names)

📌 **Example: DOGS.csv Sample Data**

ID	Name	Breed
1	Wolfie	German Shepherd
2	Fluffy	Pomeranian
3	Huggy	Labrador

---

2. Loading Data into Databases

To load CSV data into a database using **phpMyAdmin**:

1. **Upload the file** by browsing to it in phpMyAdmin
2. **Select "CSV"** from the **Format** dropdown menu
3. A section named **Format Specific Options** will appear
4. **Check the box** that specifies the first row contains column names
5. Click **Go**, and MySQL will **auto-detect headers** and populate the table

📌 **By default**, the table name is set as the **lowercase filename**

---

3. Querying Data with SQL

🔹 **Column Names with Spaces or Special Characters**

- Use **backticks (`) around column names**
- **Avoid using** single (') or double (") quotes

🔹 **Handling Long Queries**

- Break **long SQL queries** (e.g., `JOIN` or nested queries) into multiple lines for readability
- In **Python Notebooks**, use **backslashes (\)** to continue queries across multiple lines

📌 **Example: Using SQL Cell Magic in Jupyter**

```
%%sql
SELECT ID, Name, Breed
FROM dogs_table
WHERE Breed = "Labrador"
```

💡 **No need for a backslash when using cell magic (`%%sql`)**

---

4. Querying Databases in Python

🔹 Use the `read_sql()` method from the **Pandas library** to query databases in Python

🔹 Save queries in a **variable** for easier execution

📌 **Example: Running a SQL Query in Python**

```
import pandas as pd
import sqlite3

conn = sqlite3.connect("dogs.db")  # Connect to database
query_statement = "SELECT ID, Name, Breed FROM dogs_table"
df = pd.read_sql(query_statement, conn)  # Run query
print(df.head())  # Display first rows
```

💡 **Handling Quotes in Queries**

- If using **double quotes (")** in SQL (e.g., for column names), enclose the query in **single quotes (')** in Python
- To use **single quotes (')** inside the query (e.g., in a `WHERE` clause), use a **backslash (\)** as an escape character

---

5. Restricting Query Results

- Large datasets may contain **millions of rows**
- Avoid using `SELECT *` when testing or sampling data
- Use the **LIMIT** clause to **retrieve only a subset of rows**

📌 **Example: Retrieving Limited Rows**

```
SELECT * FROM census_data LIMIT 3;
```

---

6. Summary

🚀 Now you can:
✓ **Understand** how datasets are stored in CSV format
✓ **Load CSV files** into databases using phpMyAdmin
✓ **Query databases** while handling special cases like column names and long queries
✓ **Use Python's Pandas library** to fetch database records
✓ **Optimize queries** using the **LIMIT** clause to retrieve small samples efficiently

---

Getting Table and Column Details

At the end of this guide, you will be able to:
✅ **Retrieve** a list of tables in a database
✅ **Extract** column attributes from database tables

---

1. Getting a List of Tables

- Databases often contain **multiple tables**, making it hard to remember their exact names
- Most database systems provide **system/catalog tables** that store metadata about tables

📌 **Common System Tables for Listing Tables**

Database	System Table/Command
DB2	`SYSCAT.TABLES`
SQL Server	`INFORMATION_SCHEMA.TABLES`
SQLite3	`sqlite_master`
MySQL	`SHOW TABLES`

📌 Example: Retrieve Tables in SQLite3

```
SELECT name FROM sqlite_master WHERE type='table';
```

📌 Example: Retrieve Tables in MySQL

```
SHOW TABLES;
```

---

2. Extracting Column Information

- Each table has **column attributes** such as **name, type, and constraints**
- Different databases provide different ways to retrieve column details

📌 Getting Column Attributes in SQLite3

```
PRAGMA table_info(table_name);
```

📌 Getting Column Attributes in MySQL

```
DESCRIBE table_name;
```

---

3. Summary

🚀 Now you can:
✓ **Retrieve the list of tables** in a database using system tables or commands
✓ **Extract column attributes** using appropriate database-specific commands
✓ **Use SQLite3's `PRAGMA table_info(table_name)` or MySQL's `DESCRIBE table_name`** to view table structures

---

Module six

Views, Stored Procedures, and Transactions

Views

1. What is a View?

- A **view** is an alternative way of representing data from one or more tables or views.
- It **does not store data** but provides a **named specification** of a result set.
- A view behaves like a **virtual table** and can be queried in the same way as a table.

🔹 You can run **INSERT, UPDATE, and DELETE** queries on a view, which modify the underlying tables.

---

2. When to Use a View?

📌 **Common Use Cases:**
✅ **Data Security** – Restrict access to sensitive data (e.g., omit salary, birth date).
✅ **Data Combination** – Merge multiple tables meaningfully.
✅ **Simplified Access** – Provide controlled access to data without exposing full tables.
✅ **Customized Data Presentation** – Show only relevant data for specific processes.

📌 **Example:**
A view of the **Employees** table could exclude sensitive information, only showing:

- **Employee ID**
- **Name**
- **Address**
- **Job ID**
- **Manager ID**
- **Department ID**

---

3. Creating a View

🔹 The `CREATE VIEW` statement defines a **named virtual table** based on a `SELECT` query.
🔹 The **view definition is stored**, but the data remains in the base table.
🔹 You can apply conditions using the `WHERE` clause.

📌 **Syntax for Creating a View:**

```
CREATE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

📌 **Example:**
Creating a view `EMPINFO` to display only employees with `MANAGER_ID = 30002`:

```
CREATE VIEW EMPINFO AS
SELECT Employee_ID, Name, Job_ID, Manager_ID, Department_ID
FROM Employees
WHERE MANAGER_ID = 30002;
```

📌 **Querying a View:**

```
SELECT * FROM EMPINFO;
```

---

4. Deleting a View

- To remove a view completely, use the `DROP VIEW` statement.

📌 **Example:**

```
DROP VIEW EMPINFO;
```

---

5. Summary

🚀 Now you know that:
✓ **Views provide an alternate way of accessing data** without modifying base tables.
✓ **They can include specific columns** from multiple tables or existing views.
✓ **Views are dynamic** – the definition is stored, but the data comes from base tables.
✓ **`CREATE VIEW` is used to define views**, and **`DROP VIEW` removes them**.

Stored Procedures

1. What is a Stored Procedure?

- A **Stored Procedure** is a set of SQL statements stored and executed on the **database server**.
- Instead of sending multiple SQL queries from the **client** to the **server**, you encapsulate them into a stored procedure and execute them with a **single call**.
- Stored procedures can be written in **SQL, PL/SQL, Java, C**, and other languages.
- They can **accept parameters, perform CRUD operations (Create, Read, Update, Delete), and return results** to the client.

📌 **Example Use Case:**
A stored procedure can be used to update an employee's salary based on their performance rating instead of writing multiple `UPDATE` statements.

---

2. Benefits of Using Stored Procedures

✅ **Reduced Network Traffic** – Instead of sending multiple queries, you send **one request** to execute multiple statements.
✅ **Improved Performance** – Execution happens **on the server**, and only the **final result** is sent back to the client.
✅ **Code Reusability** – Multiple applications can use the **same stored procedure**

instead of writing the same SQL logic repeatedly.
✅ **Increased Security** –

- Client-side developers do **not** need access to full table/column details.
- **Validation logic** can be implemented at the **server level** before inserting data.

🚨 **Note:**
SQL is **not** a full-fledged programming language. Avoid writing all **business logic** inside stored procedures!

---

3. How to Create a Stored Procedure?

📌 Syntax for Creating a Stored Procedure:

```
DELIMITER $$

CREATE PROCEDURE procedure_name (parameters)
LANGUAGE SQL
BEGIN
    -- SQL Statements
END $$

DELIMITER ;
```

📌 Example: Updating Employee Salary Based on Rating

```
DELIMITER $$

CREATE PROCEDURE update_sal (IN emp_id INT, IN rating INT)
LANGUAGE SQL
BEGIN
    IF rating = 1 THEN
        UPDATE Employees SET salary = salary * 1.10 WHERE Employee_ID
= emp_id;
    ELSE
        UPDATE Employees SET salary = salary * 1.05 WHERE Employee_ID
= emp_id;
    END IF;
END $$

DELIMITER ;
```

🔹 Explanation:

- `DELIMITER $$` – Changes the **end statement delimiter** to `$$` (since `;` is used inside the procedure).
- `CREATE PROCEDURE update_sal` – Defines the stored procedure with **parameters** (`emp_id`, `rating`).
- **Logic:** If rating = 1, increase salary by **10%**, otherwise increase by **5%**.
- `DELIMITER ;` – Resets the delimiter to **semicolon (;)**.

---

4. How to Call a Stored Procedure?

📌 **Syntax:**

```
CALL procedure_name(parameters);
```

📌 **Example:** Calling `update_sal` procedure for Employee **ID = 101**, Rating = **1**

```
CALL update_sal(101, 1);
```

- This executes the **salary update logic** for Employee **101**.

---

5. Deleting a Stored Procedure

📌 **Syntax:**

```
DROP PROCEDURE procedure_name;
```

📌 **Example:**

```
DROP PROCEDURE update_sal;
```

---

6. Summary

🚀 Now you know that:
✓ **Stored Procedures** are a set of SQL statements executed **on the server**.
✓ They **improve performance** by reducing network traffic and reusing code.
✓ They **increase security** by restricting direct table access.
✓ **`CREATE PROCEDURE`** is used to define stored procedures, and they are called using **`CALL`**.
✓ **`DROP PROCEDURE`** removes a stored procedure.

ACID Transactions

1. What is a Transaction?

- A **transaction** is a **unit of work** in SQL that consists of **one or more SQL statements**.
- It must either **complete successfully (commit)** or **fail completely (rollback)** to

maintain database integrity.

🔹 If a transaction is **partially completed**, it could lead to **inconsistent data**, which is why we use ACID transactions.

📌 **Example Use Case:**
When buying a product online using a **bank card**, multiple updates occur:

- The **customer's account** is debited.
- The **store's account** is credited.
- The **inventory stock** is updated.
- A **purchase receipt** is generated.

❌ **If any of these steps fail, the whole transaction must be rolled back.**

---

2. What is an ACID Transaction?

🔹 ACID ensures that transactions are **reliable and consistent** even in cases of system crashes or failures.

ACID Properties

Property	Description
Atomicity	All operations must be completed successfully, or none at all.
Consistency	The database must remain in a **valid** state before and after the transaction.
Isolation	Transactions must not interfere with each other while executing.
Durability	Once committed, changes **must persist** even after a system crash.

📌 **Example:**
If Rose buys boots for **$200**, the following SQL statements execute:

```
UPDATE Accounts SET balance = balance - 200 WHERE user = 'Rose';
UPDATE Store SET balance = balance + 200 WHERE name = 'ShoeShop';
UPDATE Inventory SET stock = stock - 1 WHERE product = 'Boots';
```

🔹 If any of these statements **fail**, the entire transaction **must be rolled back**.

---

3. Managing ACID Transactions

Starting a Transaction

- In **DB2 on Cloud**, transactions begin **implicitly** when an SQL command is issued.
- In other databases, you may explicitly start a transaction using:

```
BEGIN TRANSACTION;
```

Committing a Transaction

- If **all operations succeed**, use **COMMIT** to **save changes permanently**:

```
COMMIT;
```

- Ensures that changes are **stored** and cannot be undone.

Rolling Back a Transaction

- If any operation **fails**, use **ROLLBACK** to undo all changes:

```
ROLLBACK;
```

- Restores the database to its **previous stable state** before the transaction began.

📌 **Example: Handling an ACID Transaction**

```
BEGIN TRANSACTION;

UPDATE Accounts SET balance = balance - 200 WHERE user = 'Rose';
UPDATE Store SET balance = balance + 200 WHERE name = 'ShoeShop';
UPDATE Inventory SET stock = stock - 1 WHERE product = 'Boots';

IF @@ERROR <> 0
    ROLLBACK; -- If any error occurs, undo all changes
ELSE
    COMMIT;   -- Otherwise, save all changes
```

- If Rose **doesn't have enough balance**, the transaction **rolls back**.
- If all updates **succeed**, the transaction **commits**.

---

4. Using ACID Transactions in Programming Languages

- SQL commands can be executed from programming languages using **database-specific APIs**:

Language	Database API
Java	JDBC (Java Database Connectivity)
Python	`ibm_db`, `sqlite3`, `psycopg2` (PostgreSQL)
C	Embedded SQL (`EXEC SQL COMMIT;`)

📌 Example: Python ACID Transaction

```
import sqlite3

conn = sqlite3.connect('bank.db')
cursor = conn.cursor()

try:
    cursor.execute("UPDATE Accounts SET balance = balance - 200 WHERE
user = 'Rose'")
    cursor.execute("UPDATE Store SET balance = balance + 200 WHERE
name = 'ShoeShop'")
    cursor.execute("UPDATE Inventory SET stock = stock - 1 WHERE
product = 'Boots'")

    conn.commit()  # Commit if all updates succeed
    print("Transaction Successful")
except:
    conn.rollback()  # Rollback if any error occurs
    print("Transaction Failed")
```

- Ensures that **either all operations complete successfully or none at all**.

---

5. Summary

🚀 Now you know that:
✓ A **transaction** is a unit of work that consists of **one or more SQL statements**.
✓ **ACID transactions** ensure reliability by following **Atomicity, Consistency, Isolation, and Durability**.
✓ **COMMIT** saves changes permanently, while **ROLLBACK** undoes all changes.
✓ Transactions can be managed using **SQL commands** and implemented in **programming languages**.

Lesson's Summary

- Views are a dynamic mechanism for presenting data from one or more tables.A transaction represents a complete unit of work, which can be one or more SQL statements.
- An ACID transaction is one where all the SQL statements must complete successfully, or none at all.
- A stored procedure is a set of SQL statements that are stored and executed on the database server, allowing you to send one statement as an alternative to sending multiple statements.
- You can write stored procedures in many different languages like SQL PL, PL/SQL, Java, and C.

JOIN Statements

Join Overview

1. What is a JOIN?

🔹 A **JOIN** is used to **combine rows** from **two or more tables** based on a **common column**.
🔹 The common column is usually a **primary key** in one table and a **foreign key** in another.
🔹 Using **JOINs**, we can retrieve related data stored in **separate tables** within a relational database.

📌 **Example Use Case:**
Consider a **library database** with the following tables:

- **Borrower**: Stores information about people borrowing books.
- **Loan**: Tracks which books have been borrowed.
- **Copy**: Stores individual copies of books.

If we want to know **which borrower has which book copy on loan**, we need to extract data from **three tables**:

- **Borrower → Loan → Copy**
- This is done using **JOINs** on common columns.

---

2. Understanding Primary Keys & Foreign Keys in JOINs

🔹 A **Primary Key (PK)** uniquely identifies a record in a table.
🔹 A **Foreign Key (FK)** is a column in one table that **references** the Primary Key of another table.
🔹 JOINs use these keys to **establish relationships** between tables.

📌 **Example:**

Table	Primary Key (PK)	Foreign Key (FK)
Borrower	borrower_id	-
Loan	loan_id	borrower_id (FK)
Copy	copy_id	-

If we want to find which **borrower has borrowed which copy of a book**, we JOIN:

1. **Borrower** table with **Loan** table using `borrower_id`.
2. **Loan** table with **Copy** table using `copy_id`.

---

3. Types of JOINs in SQL

A. INNER JOIN (Most Common)

🔹 Returns **only matching rows** where the common column has values in both tables.

```
SELECT Borrower.name, Loan.loan_id
FROM Borrower
INNER JOIN Loan ON Borrower.borrower_id = Loan.borrower_id;
```

✅ Shows only borrowers who **have taken a loan**.

B. OUTER JOINs (Includes Non-Matching Rows)

1. LEFT JOIN (Left Outer Join)

🔹 Returns **all rows from the left table**, and matching rows from the right table.
🔹 If no match is found, `NULL` values appear for right table columns.

```
SELECT Borrower.name, Loan.loan_id
FROM Borrower
LEFT JOIN Loan ON Borrower.borrower_id = Loan.borrower_id;
```

✅ Shows **all borrowers**, even those who **haven't borrowed books**.

2. RIGHT JOIN (Right Outer Join)

🔹 Returns **all rows from the right table**, and matching rows from the left table.
🔹 If no match is found, `NULL` values appear for left table columns.

```
SELECT Borrower.name, Loan.loan_id
FROM Borrower
RIGHT JOIN Loan ON Borrower.borrower_id = Loan.borrower_id;
```

✅ Shows **all loan records**, even if some **borrowers are missing**.

3. FULL JOIN (Full Outer Join)

🔹 Returns **all rows from both tables**, with `NULL` for unmatched rows.

```
SELECT Borrower.name, Loan.loan_id
FROM Borrower
FULL JOIN Loan ON Borrower.borrower_id = Loan.borrower_id;
```

✅ Includes **all borrowers and all loans**, even if there's no match.

---

4. Joining Multiple Tables

🔹 You can **JOIN more than two tables** by adding additional JOINs.

📌 **Example: Find which borrower has borrowed which book copy**

```
SELECT Borrower.name, Copy.book_id
FROM Borrower
INNER JOIN Loan ON Borrower.borrower_id = Loan.borrower_id
INNER JOIN Copy ON Loan.copy_id = Copy.copy_id;
```

✅ First **JOIN Borrower & Loan**, then **JOIN Loan & Copy**.

---

5. Summary

🚀 Now you know that:
✓ The **JOIN operator** is used to combine rows from two or more tables.
✓ Tables are related using **Primary Keys** and **Foreign Keys**.
✓ **INNER JOIN** returns only **matching rows**.
✓ **OUTER JOINs** (LEFT, RIGHT, FULL) return additional **non-matching rows**.
✓ **Multiple JOINs** can be used to combine data from **three or more tables**.

Inner Join

1. What is an INNER JOIN?

🔹 An **INNER JOIN** is a type of **JOIN** that returns **only matching rows** between two tables.
🔹 The matching occurs based on a **common column** that links both tables.
🔹 This common column is usually a **Primary Key (PK)** in one table and a **Foreign Key (FK)** in another.

📌 **Example Use Case:**
Imagine you want to retrieve a list of all people who are **borrowing books** and the **date of their loan**.
You need data from two tables:

- **Borrower** (holds borrower details)
- **Loan** (records loan transactions)

To get the required data, you **JOIN** these tables using the **borrower_id** column.

---

2. When to Use an INNER JOIN?

Use **INNER JOIN** when you need:
✓ Only the **matching records** between two tables.
✓ To filter out **rows that do not have corresponding matches** in both tables.
✓ To retrieve related data stored across **multiple tables**.

📌 **Example:**

- Find students who have enrolled in at least one course.
- Get employees who are assigned to a department.
- List customers who have made a purchase.

---

3. INNER JOIN Syntax

🔹 The basic syntax of an **INNER JOIN** in SQL is:

```
SELECT column_names
FROM table1
INNER JOIN table2
ON table1.common_column = table2.common_column;
```

🔹 You can **alias** table names to make the query shorter and more readable.

---

4. INNER JOIN Example

📌 **Retrieve a list of borrowers with loan details**

```
SELECT B.borrower_id, B.last_name, B.country, L.loan_date
FROM Borrower AS B
INNER JOIN Loan AS L
ON B.borrower_id = L.borrower_id;
```

Explanation

1️⃣ `FROM Borrower AS B` → The **left table** (Borrower) is given alias **B**.
2️⃣ `INNER JOIN Loan AS L` → The **right table** (Loan) is given alias **L**.
3️⃣ `ON B.borrower_id = L.borrower_id` → The **JOIN condition** links Borrower and Loan tables using **borrower_id**.
4️⃣ **Only matching records** (borrowers who have taken loans) appear in the result.

Sample Result Set

borrower_id	last_name	country	loan_date
101	Smith	USA	2024-01-10
102	Johnson	UK	2024-01-12
104	Lee	Canada	2024-02-01

❌ Borrowers **without loans** are **excluded** from the result.

---

5. Key Takeaways

✓ **INNER JOIN** returns only rows with **matching values** in both tables.
✓ Rows that **do not have matches** in the other table are **excluded**.
✓ The **common column** is usually a **Primary Key** in one table and a **Foreign Key** in another.
✓ **Aliases** (`AS B`, `AS L`) make queries more readable.

Outer Joins

1. What is an OUTER JOIN?

- Like an **INNER JOIN**, an **OUTER JOIN** returns rows with **matching values** between tables.
- However, an **OUTER JOIN** also **includes unmatched rows** from one or both tables.
- SQL offers three types of **OUTER JOINs**:

JOIN Type	Returns
LEFT OUTER JOIN	All rows from the **left** table + matching rows from the right table
RIGHT OUTER JOIN	All rows from the **right** table + matching rows from the left table
FULL OUTER JOIN	All rows from **both** tables (matched + unmatched)

---

2. Types of OUTER JOINs

A. LEFT OUTER JOIN (LEFT JOIN)

✅ Returns **all rows** from the **left** table.
✅ Includes **matching rows** from the **right** table.
✅ If no match is found, columns from the **right** table show `NULL`.

📌 **Example Use Case:**

- List **all borrowers**, including those **who haven't borrowed books**.

```
SELECT B.borrower_id, B.last_name, B.country, L.loan_date
FROM Borrower AS B
LEFT JOIN Loan AS L
ON B.borrower_id = L.borrower_id;
```

Example Result

borrower_id	last_name	country	loan_date
101	Smith	USA	2024-01-10
102	Johnson	UK	2024-01-12
103	Peters	Canada	NULL
104	Lee	Germany	2024-02-01
105	Wong	China	NULL

❌ **Borrowers without loans have `NULL` loan dates**.

B. RIGHT OUTER JOIN (RIGHT JOIN)

✅ Returns **all rows** from the **right** table.
✅ Includes **matching rows** from the **left** table.
✅ If no match is found, columns from the **left** table show `NULL`.

📌 **Example Use Case:**

- List **all loan records**, including loans **without known borrowers**.

```
SELECT L.borrower_id, L.loan_date, B.last_name, B.country
FROM Borrower AS B
RIGHT JOIN Loan AS L
ON B.borrower_id = L.borrower_id;
```

Example Result

borrower_id	loan_date	last_name	country
101	2024-01-10	Smith	USA
102	2024-01-12	Johnson	UK
104	2024-02-01	Lee	Germany
999	2024-03-01	NULL	NULL

❌ **The last row (`999`) has no borrower information (`NULL`).**
❌ **This might indicate an issue where a book was loaned to an unknown person**.

C. FULL OUTER JOIN (FULL JOIN)

✅ Returns **all rows** from **both** tables.
✅ Includes **matching rows** and **non-matching rows** from both tables.
✅ If no match exists, columns from the other table show `NULL`.

📌 **Example Use Case:**

- Get **all borrowers and all loan records**, including unmatched ones.

```
SELECT B.borrower_id, B.last_name, B.country, L.loan_date
FROM Borrower AS B
FULL JOIN Loan AS L
ON B.borrower_id = L.borrower_id;
```

Example Result

borrower_id	last_name	country	loan_date
101	Smith	USA	2024-01-10
102	Johnson	UK	2024-01-12
103	Peters	Canada	NULL
104	Lee	Germany	2024-02-01
105	Wong	China	NULL
999	NULL	NULL	2024-03-01

❌ **Rows with `NULL` values indicate unmatched data**.
✓ **Useful when you need a comprehensive view of both tables**.

---

3. Key Takeaways

✓ **LEFT JOIN** → Returns **all** left table rows + **matching** right table rows.
✓ **RIGHT JOIN** → Returns **all** right table rows + **matching** left table rows.
✓ **FULL JOIN** → Returns **all** rows from **both** tables (matched + unmatched).

✅ **Use LEFT JOIN** when you want **all records from the left table**, even if they have no match.
✅ **Use RIGHT JOIN** when you want **all records from the right table**, even if they have no match.
✅ **Use FULL JOIN** when you want **all records from both tables**, regardless of a match.

Lesson's Summary:

- A join combines the rows from two or more tables based on a relationship between certain columns in these tables.
- To combine data from three or more different tables, you simply add new joins to the SQL statement.
- There are two types of joins: inner join and outer join; and three types of outer joins: left outer join, right outer join, and full outer join.
- The most common type of join is the inner join, which matches the results from two tables and returns the selected elements from each table, only where corresponding elements in both the tables are the same.
- You can use an alias as shorthand for a table or column name.
